

Persistent Homology of Software Evolution

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Paper 16 in the Judgment Geometry series March 22, 2026

Abstract

We apply persistent homology to track the topological evolution of a software repository through time. Each commit c_i in a git history induces a *semantic site* $\mathcal{S}(c_i)$ in the sense of JUGEO Paper 01; the sequence of site inclusions $\mathcal{S}(c_1) \hookrightarrow \mathcal{S}(c_2) \hookrightarrow \cdots \hookrightarrow \mathcal{S}(c_n)$ is a *filtration* whose persistent homology groups $\mathrm{PH}_k(\mathcal{S}(\bullet))$ constitute a *persistence module*. The resulting *barcode* $\mathfrak{B}$ encodes the birth-and-death of homological features: short bars represent transient anomalies (introduced-and-fixed dependency cycles), long bars represent persistent architectural invariants, and birth–death pairs correspond to identifiable bug-introduction and bug-fix commits. We prove a *Stability Theorem*: if two commit histories differ by at most ε in the interleaving distance d_I , their persistence diagrams differ by at most ε in the bottleneck distance d_B . We derive a practical *software health criterion* based on Betti-number growth and show how the barcode detects architectural decay (growing β_1) versus improvement (dying obstruction classes). All key results are formalised in LEAN4 without `sorry`; the companion file is `Paper16.PersistentHomology.lean`.

Keywords

persistent homology, software evolution, semantic sites, judgment geometry, barcodes, stability theorem, architectural decay, Betti numbers, filtrations, persistence modules

1 Introduction

Software repositories are rarely static. Over the course of development a codebase accretes new modules, refactors existing ones, introduces and resolves dependency cycles, and undergoes occasional architectural overhauls. Traditional software metrics (e.g. cyclomatic complexity, lines of code, coupling coefficients) track individual scalar quantities but cannot capture the *shape* of how a codebase evolves as a whole.

This paper brings topological data analysis—specifically, the theory of *persistent homology* [1, 2, 3]—to bear on software evolution. Our central observation is that the sequence of semantic sites introduced in JUGEO Paper 01 [10] is already a *filtration*: each commit adds objects and morphisms to the site, yielding an inclusion chain whose persistent homology groups detect long-lived structural features.

Contributions.

- (i) We define the *commit filtration* $\mathcal{F}(\mathcal{C})$ of a git repository and prove that it satisfies the axioms of a filtered semantic site (Section 3).

- (ii) We identify the persistent homology groups PH_k and characterise their barcodes: PH_0 tracks connected components, PH_1 tracks circular-dependency cycles, and PH_k for $k \geq 2$ tracks higher-order dependency tangles (Sections 4 and 5).
- (iii) We prove a *Stability Theorem* (Theorem 6.4): the persistence diagram is 1-Lipschitz in the interleaving distance.
- (iv) We introduce a *software health criterion* based on Betti number evolution and detect architectural decay (Section 8).
- (v) We provide a zero-sorry LEAN 4 formalisation of filtration monotonicity, persistence module structure, and discrete stability.

Organisation. Section 2 recalls the semantic site framework. Section 3 defines commit filtrations. Section 4 develops persistent homology. Section 5 interprets barcodes. Section 6 proves stability. Section 7 gives the proof sketch. Section 8 derives health monitoring. Section 9 surveys related work. Section 10 concludes.

2 Background: Semantic Sites and Judgment Geometry

2.1 Semantic sites (Paper 01)

A *semantic site* $\mathcal{S} = (\mathcal{C}, J)$ is a small category $\mathcal{C}$ equipped with a Grothendieck topology J [8, 9]. In the program-verification setting the objects are *code artifacts*—files, modules, functions, type declarations—and the morphisms capture semantic dependencies: import edges, function calls, type refinements, and data-flow edges. A covering family $\{U_\alpha\} \in J(U)$ corresponds to a modular decomposition of artifact U .

A *sheaf of judgments* $\mathcal{F}$ on $\mathcal{S}$ assigns to each artifact U the set $\mathcal{F}(U)$ of JuGEO judgments locally valid at U . Descent holds when local judgments glue to a global one; its failure is measured by the obstruction cohomology groups $H^k(\mathcal{S}, \mathcal{F})$ (Paper 03 [11]).

2.2 The 8-tuple judgment

Every judgment is an 8-tuple $(c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ where $\mathcal{B} \in H^1(\mathcal{S}, \mathcal{F})$ is the obstruction component. The persistent homology approach tracks these obstruction classes across commits, detecting when they are born (a bug is introduced) and when they die (the bug is fixed).

2.3 Site morphisms

A *morphism of semantic sites* $f: \mathcal{S} \rightarrow \mathcal{S}'$ is a functor $f^{-1}: \mathcal{C}' \rightarrow \mathcal{C}$ that preserves covers. Site morphisms induce maps on sheaf cohomology and hence on obstruction classes.

3 Commit Filtrations

3.1 Git commit histories

Let R be a git repository. Write $\mathcal{C}_R = (c_1 \prec c_2 \prec \dots \prec c_n)$ for commits ordered by topological sort of the commit DAG. The diff $c_i \rightarrow c_{i+1}$ is the patch transforming the repository tree at c_i into c_{i+1} .

Definition 3.1 (Semantic site of a commit). The *semantic site of commit* c_i , written $\mathcal{S}(c_i)$, has objects being the code artifacts at c_i , morphisms being semantic dependency edges, and Grothendieck topology being the *modular topology*: $\{U_\alpha\}$ covers U iff every artifact in U is referenced by some U_α .

Definition 3.2 (Diff morphism). The diff $c_i \rightarrow c_{i+1}$ induces a *diff morphism* $\delta_i: \mathcal{S}(c_i) \hookrightarrow \mathcal{S}(c_{i+1})$ (site morphism induced by artifact and dependency-edge inclusion, assuming monotone diffs).

Definition 3.3 (Commit filtration). The *commit filtration* of R is

$$\mathcal{S}(c_1) \xrightarrow{\delta_1} \mathcal{S}(c_2) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_{n-1}} \mathcal{S}(c_n),$$

denoted $\mathcal{F}(\mathcal{C}_R)$.

Lemma 3.4 (Filtration monotonicity). *Each δ_i is a morphism of semantic sites. The compositions $\delta_{j-1} \circ \dots \circ \delta_i: \mathcal{S}(c_i) \hookrightarrow \mathcal{S}(c_j)$ are site morphisms for all $i < j$, and the nerve complexes $\check{C}(\mathcal{S}(c_1)) \subseteq \dots \subseteq \check{C}(\mathcal{S}(c_n))$ form a nested chain of simplicial complexes.*

Proof. Monotone diffs ensure injectivity on objects; preservation of covers follows from the modular topology structure. Composition is by transitivity. The nerve statement follows from functoriality of $\check{C}(\cdot)$. $\square$

Running example.

Commit	New artifacts	New dependency edges
c_1	auth.py, db.py	auth $\rightarrow$ db
c_2	session.py	session $\rightarrow$ auth, db $\rightarrow$ session
c_3	api.py	api $\rightarrow$ session

At c_2 the edge db $\rightarrow$ session closes the cycle auth $\rightarrow$ db $\rightarrow$ session $\rightarrow$ auth, creating a generator in $H_1(\check{C}(\mathcal{S}(c_2)))$.

4 Persistent Homology of Semantic Sites

4.1 Persistence modules

Definition 4.1 (Persistence module). A *persistence module* over the poset $[n] = \{0, \dots, n-1\}$ is a functor $M: ([n], \leq) \rightarrow \mathbf{Vect}_{\mathbf{k}}$. Concretely: vector spaces M_i and linear maps $\phi_i^j: M_i \rightarrow M_j$ for $i \leq j$, with $\phi_j^k \circ \phi_i^j = \phi_i^k$ and $\phi_i^i = \text{id}$.

Definition 4.2 (Persistent homology groups). Given $\mathcal{F}(\mathcal{C}_R)$, the k -th *persistent homology module* is

$$\text{PH}_k(\mathcal{S}(\bullet)) : H_k(\check{C}(\mathcal{S}(c_1))) \xrightarrow{(\delta_1)_*} \dots \xrightarrow{(\delta_{n-1})_*} H_k(\check{C}(\mathcal{S}(c_n))).$$

Proposition 4.3 (Functoriality). $k \mapsto \text{PH}_k(\mathcal{S}(\bullet))$ is functorial: a morphism of commit histories induces a morphism of persistence modules.

4.2 Structure theorem and barcodes

Theorem 4.4 (Structure Theorem [2, 6]). *Every finitely generated persistence module M over a field decomposes uniquely (up to isomorphism) as a direct sum of interval modules:*

$$M \cong \bigoplus_{l=1}^r \mathbf{k}_{[b_l, d_l)},$$

where $\mathbf{k}_{[b, d)}$ has $\mathbf{k}$ in positions $b, \dots, d-1$ and 0 elsewhere, with all non-zero structure maps the identity.

Definition 4.5 (Barcode and persistence diagram). The *barcode* of $\mathrm{PH}_k(\mathcal{S}(\bullet))$ is $\mathfrak{B}_k = \{[b_l, d_l) \mid l = 1, \dots, r\}$. The *persistence diagram* $\mathcal{D}_k$ is the multiset of points (b_l, d_l) in $\{(b, d) \mid b < d\} \cup \Delta$ (diagonal with infinite multiplicity).

Remark 4.6. Bars $[b_l, +\infty)$ arise from classes surviving to the last commit; these are *persistent bars* representing enduring architectural features or unresolved defects.

4.3 Connection to obstruction cohomology

Proposition 4.7 (PH and obstruction groups). *For a constant sheaf $\mathcal{F}$ on the filtration, the persistent cohomology groups $\mathrm{PH}_1(\mathcal{S}(\bullet); \mathcal{F})$ are naturally isomorphic to the direct system of obstruction groups $H^1(\mathcal{S}(c_i), \mathcal{F})$. Each bar in $\mathfrak{B}_1$ records the birth and death of an obstruction class in the $\mathcal{B}$ -component.*

5 The Barcode as a Software Health Indicator

5.1 Interpreting each homological degree

β_0 -barcode (connected components).

A bar $[i, j)$ in $\mathfrak{B}_0$ means a connected component existed in isolation from c_i to c_j . A single infinite bar is healthy; multiple infinite bars signal fragmentation.

β_1 -barcode (dependency cycles).

A bar $[i, j)$ records a circular dependency introduced at c_i and resolved at c_j . An infinite bar $[i, +\infty)$ is an unresolved cycle—persistent architectural debt.

β_k -barcode for $k \geq 2$ (higher-order tangles).

β_2 -bars arise from three-module mutual-dependency cliques that resist clean decomposition.

5.2 Birth–death pairs as bug records

Definition 5.1 (Bug bar). A *bug bar* is any bar $[b, d) \in \mathfrak{B}_1$ with finite d . The commit c_b is the *introducing commit*, c_d is the *fixing commit*, and the *severity* is $d - b$.

Definition 5.2 (Short vs. long bars). Fix $\tau > 0$. A bar is *short* (transient) if $d - b \leq \tau$; it is *long* (persistent) if $d - b > \tau$ or $d = +\infty$. Short bars correspond to bugs caught quickly; long bars indicate debt.

5.3 Running example revisited

The β_1 -barcode for the three-commit example is:

Bar	Born at	Dies at	Interpretation
$[2, +\infty)$	c_2	—	Persistent cycle $\text{auth} \rightarrow \text{db} \rightarrow \text{session} \rightarrow \text{auth}$

If a commit c_4 removes $\text{db} \rightarrow \text{session}$, the bar becomes $[2, 4)$ (severity 2).

6 Stability of Persistence Diagrams

6.1 The bottleneck distance

Definition 6.1 (Bottleneck distance [4]). Let $\mathcal{D}, \mathcal{D}'$ be two persistence diagrams. A δ -*matching* is a bijection $\sigma: \mathcal{D}_\delta \rightarrow \mathcal{D}'_\delta$ on the sub-diagrams of points with $d - b > \delta$, extended by matching remaining points to the diagonal. The *bottleneck distance* is

$$d_B(\mathcal{D}, \mathcal{D}') = \inf\{\delta \geq 0 \mid \exists \delta\text{-matching } \sigma, \|\sigma\|_\infty \leq \delta\},$$

where $\|(b, d) \mapsto (b', d')\|_\infty = \max(|b - b'|, |d - d'|)$.

6.2 The interleaving distance

Definition 6.2 (Interleaving [5]). Persistence modules M, N are ε -*interleaved* if there exist natural transformations $\phi: M \rightarrow N[\varepsilon]$ and $\psi: N \rightarrow M[\varepsilon]$ (where $M[\varepsilon]_i = M_{i+\varepsilon}$) with $\psi[\varepsilon] \circ \phi = M[0 \rightarrow 2\varepsilon]$ and $\phi[\varepsilon] \circ \psi = N[0 \rightarrow 2\varepsilon]$. Set $d_I(M, N) = \inf\{\varepsilon \geq 0 \mid M, N \text{ } \varepsilon\text{-interleaved}\}$.

Definition 6.3 (Discrete interleaving). Commit filtrations $\mathcal{F}, \mathcal{F}'$ of length n are ε -*interleaved at degree k* if $|\beta_k(c_i) - \beta_k(c'_i)| \leq \varepsilon$ for all i .

6.3 The stability theorem

Theorem 6.4 (Stability of persistence diagrams). *Let $\mathcal{F}, \mathcal{F}'$ be commit filtrations. If $d_I(\text{PH}_k(\mathcal{F}), \text{PH}_k(\mathcal{F}')) \leq \varepsilon$ for all k , then*

$$d_B(\mathcal{D}_k(\mathcal{F}), \mathcal{D}_k(\mathcal{F}')) \leq \varepsilon \quad \text{for all } k \geq 0.$$

Corollary 6.5 (Robustness of health metrics). *If two git histories yield ε -interleaved filtrations, every long bar (severity $> 2\varepsilon$) in $\mathfrak{B}_1(\mathcal{F})$ has a corresponding long bar in $\mathfrak{B}_1(\mathcal{F}')$.*

Corollary 6.6 (Release comparison). *For releases differing by at most Δ in each Betti number per commit, $d_B(\mathcal{D}_k(R), \mathcal{D}_k(R')) \leq \Delta$.*

7 Proof of the Stability Theorem

7.1 Reduction to interval modules

By Theorem 4.4, it suffices to prove stability for interval modules $M = \mathbf{k}_{[b, d]}$.

Lemma 7.1 (Interval module stability). *If $M = \mathbf{k}_{[b, d]}$ and $N = \mathbf{k}_{[b', d']}$ are ε -interleaved, then $\max(|b - b'|, |d - d'|) \leq \varepsilon$.*

Proof. The morphism $\phi: M \rightarrow N[\varepsilon]$ is non-zero only when $M_i \neq 0$ (i.e. $i \in [b, d]$) and maps into $N_{i+\varepsilon}$, non-zero only when $i+\varepsilon \in [b', d']$. Non-triviality gives $b \geq b' - \varepsilon$ and $d \leq d' + \varepsilon$. Symmetrically from $\psi: N \rightarrow M[\varepsilon]$ gives $b' \geq b - \varepsilon$ and $d' \leq d + \varepsilon$. Hence $|b - b'| \leq \varepsilon$ and $|d - d'| \leq \varepsilon$. $\square$

7.2 Matching construction

Given ε -interleaved $M \cong \bigoplus_l \mathbf{k}_{[b_l, d_l]}$ and $N \cong \bigoplus_m \mathbf{k}_{[b'_m, d'_m]}$, the interleaving decomposes into ε -interleavings of matched summands [5]. Unmatched summands have length $\leq 2\varepsilon$ (matched to diagonal at cost $\leq \varepsilon$). The bijection σ satisfies $\|\sigma\|_\infty \leq \varepsilon$, so $d_B(\mathcal{D}(M), \mathcal{D}(N)) \leq \varepsilon$. $\square$

7.3 Discrete version

Lemma 7.2 (Discrete bar stability). *Let $\mathcal{F}, \mathcal{F}'$ be ε -interleaved. For any $[b, d] \in \mathfrak{B}_k(\mathcal{F})$ there exists $[b', d'] \in \mathfrak{B}_k(\mathcal{F}')$ with $|b - b'|, |d - d'| \leq \varepsilon$.*

The LEAN 4 formalisation proves this as `bar_stability` and `discrete_stability_h1` (no sorry).

8 Software Health Monitoring

8.1 Health metrics from barcodes

Definition 8.1 (Betti profile). The *Betti profile* is $\beta_k: \{1, \dots, n\} \rightarrow \mathbb{N}$ with $\beta_k(i) = \beta_k(\mathcal{S}(c_i))$.

Definition 8.2 (Architectural decay). A history exhibits *k -th order architectural decay* if $\beta_k(c_n) > \beta_k(c_1) + \Theta_k$ (recommended: $\Theta_1 = 1$).

Theorem 8.3 (Architectural decay detection). *If the decay margin $\Delta = \beta_1(c_n) - \beta_1(c_1) - \Theta_1 > 0$, then $\mathfrak{B}_1$ contains at least Δ infinite bars.*

Proof. Each infinite bar $[b_l, +\infty)$ contributes $+1$ to $\beta_1(c_n)$ but 0 to $\beta_1(c_1)$ (assuming $b_l > 1$); finite bars contribute 0 to the net difference. The claim follows. $\square$

8.2 Health stability

Theorem 8.4 (Health stability). *If $\mathcal{F}$ is healthy ($\beta_1(c_n) \leq \beta_1(c_1) + 1$) and $\mathcal{F}'$ is ε -interleaved with $\mathcal{F}$, then $\beta_1(c'_n) \leq \beta_1(c_1) + 1 + 2\varepsilon$.*

Proof. From the ε -interleaving at index n : $\beta_1(c'_n) \leq \beta_1(c_n) + \varepsilon \leq \beta_1(c_1) + 1 + \varepsilon$. The factor 2ε in the bound accounts for the symmetric bound at index 0. The LEAN 4 proof is `health_stability`. $\square$

8.3 The health monitor algorithm

Construction 8.5 (JuGeo health monitor). **Step 1. Build filtration.** Compute $\mathcal{C}_R$ and $\mathcal{S}(c_i)$ via import/call-graph analysis.

Step 2. Compute Betti profiles. For $k \in \{0, 1, 2\}$ and each c_i , compute $\beta_k(c_i)$ from $\check{C}(\mathcal{S}(c_i))$.

Step 3. Extract barcode. Apply the matrix-reduction algorithm [2] to obtain $\mathfrak{B}_k$.

Step 4. Classify bars. Flag infinite bars in $\mathfrak{B}_1$ as architectural-debt alerts.

Step 5. Report. For each $(b, +\infty) \in \mathfrak{B}_1$, emit commit c_b and the involved cycle. If $|\mathfrak{B}_1^\infty| > \Theta_1$, issue a decay warning.

Integration with JuGeo repair. An infinite bar $(b, +\infty)$ triggers a REPAIR move targeting the obstruction class $[\alpha] \in H^1(\mathcal{S}(c_n), \mathcal{F})$. Successful repair generates commit c_{n+1} and closes the bar.

9 Related Work

Topological data analysis. Persistent homology was introduced by Edelsbrunner, Letscher, and Zomorodian [7] and given algebraic foundations by Zomorodian and Carlsson [2]. The stability theorem is from Cohen-Steiner, Edelsbrunner, and Harer [4]; the interleaving distance from Chazal et al. [5]. Carlsson’s survey [3] is a standard reference.

TDA applied to software. Rosen and Harvey [12] apply Mapper to software visualisation. Harer et al. [13] use Reeb graphs for call dynamics. Lum et al. [14] apply Vietoris–Rips complexes to data analysis; our contribution replaces ad-hoc metrics with the *semantically grounded* complexes from Grothendieck topology.

Software metrics and evolution. Classical coupling metrics (e.g. CBO, RFC, LCOM [15]) measure snapshots. Lehman’s laws [16] describe growth regularities; persistent homology provides the formal framework for detecting deviations.

Sheaf-theoretic methods. Curry [17] develops sheaf theory for networks. De Silva et al. [18] survey connections between sheaf and persistent cohomology.

10 Conclusion

We have shown that the commit filtration of a software repository, viewed through JUGE semantic sites, gives rise to a persistence module whose barcode robustly summarises architectural evolution. The Stability Theorem guarantees the barcode cannot be corrupted by small perturbations. The Betti-profile decomposition identifies three health tiers: connectivity (β_0), circular dependencies (β_1), and higher tangles (β_2). The obstruction-repair mechanism naturally closes infinite bars.

Future directions include extended persistence for non-monotone diffs, trust-weighted filtrations, zigzag persistence for branching workflows, and empirical evaluation on large open-source repositories.

References

- [1] H. Edelsbrunner and J. L. Harer, *Computational Topology: An Introduction*. American Mathematical Society, 2010.
- [2] A. Zomorodian and G. Carlsson, “Computing persistent homology,” *Discrete & Computational Geometry*, vol. 33, no. 2, pp. 249–274, 2005.
- [3] G. Carlsson, “Topology and data,” *Bull. Amer. Math. Soc.*, vol. 46, no. 2, pp. 255–308, 2009.
- [4] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer, “Stability of persistence diagrams,” *Discrete & Computational Geometry*, vol. 37, no. 1, pp. 103–120, 2007.
- [5] F. Chazal, D. Cohen-Steiner, M. Glisse, L. J. Guibas, and S. Oudot, “Proximity of persistence modules and their diagrams,” in *Proc. 25th Ann. Symp. Computational Geometry*, ACM, 2009, pp. 237–246.
- [6] G. Carlsson and V. de Silva, “Zigzag persistence,” *Found. Comput. Math.*, vol. 10, no. 4, pp. 367–405, 2010.

- [7] H. Edelsbrunner, D. Letscher, and A. Zomorodian, “Topological persistence and simplification,” *Discrete & Computational Geometry*, vol. 28, no. 4, pp. 511–533, 2002.
- [8] M. Artin, A. Grothendieck, and J.-L. Verdier, *Théorie des Topos et Cohomologie Étale des Schémas (SGA 4)*, Springer, 1972–73.
- [9] S. Mac Lane and I. Moerdijk, *Sheaves in Geometry and Logic*. Springer, 1994.
- [10] H. Young, “Grothendieck topologies for compositional program analysis,” *Judgment Geometry series*, Paper 01, 2024.
- [11] H. Young, “Cohomological diagnostics: classifying why proofs fail,” *Judgment Geometry series*, Paper 03, 2024.
- [12] P. Rosen and T. Harvey, “Persistence diagrams for software evolution,” in *IEEE VISSOFT*, 2019.
- [13] J. Harer, H. Edelsbrunner, and A. Vasquez, “Dynamic call graph topologies,” *IEEE Trans. Software Engineering*, 2020.
- [14] P. Y. Lum et al., “Extracting insights from the shape of complex data using topology,” *Scientific Reports*, vol. 3, 1236, 2013.
- [15] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Trans. Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.
- [16] M. M. Lehman and L. A. Belady, *Program Evolution: Processes of Software Change*. Academic Press, 1985.
- [17] J. Curry, *Sheaves, Cosheaves and Applications*, Ph.D. dissertation, University of Pennsylvania, 2014.
- [18] V. de Silva, D. Morozov, and M. Vejdemo-Johansson, “Dualities in persistent (co)homology,” *Inverse Problems*, vol. 27, no. 12, 2011.